Module Interface Specification for TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

November 13, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at https://github.com/Nour-Fawaz/MacaBoca/blob/main/pdfs/SRS/SRS.pdf

[Also add any additional symbols, abbreviations or acronyms —SS]

Contents

1 Revision History i

2 Symbols, Abbreviations and Acronyms ii

3 Introduction 1

4 Notation 1

5 Module Decomposition 1

6 MIS of [Module Name —SS] 3
- 6.1 Module . . . 3
- 6.2 Uses . . . 3
- 6.3 Syntax . . . 3
  - 6.3.1 Exported Constants . . . 3
  - 6.3.2 Exported Access Programs . . . 3
- 6.4 Semantics . . . 3
  - 6.4.1 State Variables . . . 3
  - 6.4.2 Environment Variables . . . 3
  - 6.4.3 Assumptions . . . 3
  - 6.4.4 Access Routine Semantics . . . 3
  - 6.4.5 Local Functions . . . 4

7 MIS of ML Transcription Model 4
- 7.1 Module . . . 4
- 7.2 Uses . . . 4
- 7.3 Syntax . . . 4
  - 7.3.1 Exported Constants . . . 4
  - 7.3.2 Exported Access Programs . . . 5
- 7.4 Semantics . . . 5
  - 7.4.1 State Variables . . . 5
  - 7.4.2 Environment Variables . . . 5
  - 7.4.3 Assumptions . . . 6
  - 7.4.4 Access Routine Semantics . . . 6
  - 7.4.5 Local Functions . . . 7

8 MIS of Transcription Service Interface Module 8
- 8.1 Module . . . 8
- 8.2 Uses . . . 8
- 8.3 Syntax . . . 8
  - 8.3.1 Exported Constants . . . 8
  - 8.3.2 Exported Access Programs . . . 8

8.4 Semantics . . . 10
8.4.1 State Variables . . . 10
8.4.2 Environment Variables . . . 10
8.4.3 Assumptions . . . 10
8.4.4 Access Routine Semantics . . . 10
8.4.5 Local Functions . . . 12

9 MIS of Video Recording Module **13**
9.1 Module . . . 13
9.2 Uses . . . 13
9.3 Syntax . . . 13
9.3.1 Exported Constants . . . 13
9.3.2 Exported Access Programs . . . 14
9.4 Semantics . . . 14
9.4.1 State Variables . . . 14
9.4.2 Environment Variables . . . 15
9.4.3 Assumptions . . . 15
9.4.4 Access Routine Semantics . . . 15
9.4.5 Local Functions . . . 16

10 Appendix — Generative AI Use Disclosure **17**
10.1 AI Tools Used . . . 17
10.2 How and Where Used . . . 17
10.3 Verification of AI-Generated Content . . . 17

11 Appendix — Reflection **18**

3 Introduction

The following document details the Module Interface Specifications for [Fill in your project name and description —SS]

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at .... [provide the url for your repo —SS]

4 Notation

[You should describe your notation. You can use what is below as a starting point. —SS]

The structure of the MIS for modules comes from **?**, with the addition that template modules have been adapted from **?**. The mathematical notation comes from Chapter 3 of **?**. For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1|c_2 \Rightarrow r_2|...|c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by TeLaC.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of TeLaC uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, TeLaC uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**
Hardware-Hiding	Device I/O Module
Behaviour-Hiding	Frontend Module Account System Module Video Recording Module Transcription Service Interface Module Report Generation Module Assessment Review Module MERLS Module
Software Decision	API Gateway Module Database Module Encryption Module ML Transcription Model Module

Table 1: Module Hierarchy

6 MIS of [Module Name —SS]

[Use labels for cross-referencing —SS]

[You can reference SRS labels, such as R??. —SS]

[It is also possible to use LaTeXfor hypperlinks to external documents. —SS]

6.1 Module

[Short name for the module —SS]

6.2 Uses

6.3 Syntax

6.3.1 Exported Constants

6.3.2 Exported Access Programs

Name	In	Out	Exceptions
[accessProg —SS]	-	-	-

6.4 Semantics

6.4.1 State Variables

[Not all modules will have state variables. State variables give the module a memory. —SS]

6.4.2 Environment Variables

[This section is not necessary for all modules. Its purpose is to capture when the module has external interaction with the environment, such as for a device driver, screen interface, keyboard, file, etc. —SS]

6.4.3 Assumptions

[Try to minimize assumptions and anticipate programmer errors via exceptions, but for practical purposes assumptions are sometimes appropriate. —SS]

6.4.4 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

6.4.5 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7 MIS of ML Transcription Model

The ML Transcription Model module converts an assessment audio file into timed text segments using a locally run Whisper model, flagging uncertain segments for clinician review (SR??, NFR??).

7.1 Module

MLTransModel

7.2 Uses

- Locally installed Whisper speech-to-text model
- Local file system (to read the audio file)

This module is used by the Transcription Service Interface module, which supplies the audio file and receives the segments. Storage and transmission of audio and transcripts are handled by other modules (Database, Encryption, API Gateway).

7.3 Syntax

7.3.1 Exported Constants

- CHUNK_DURATION = 30 seconds
  Length of each increment of audio fed to the model (Whisper's own processing window). Each increment yields one segment.
- CONFIDENCE_THRESHOLD = 0.5
  Segments with confidence below this value are flagged for review.

- SUPPORTED_AUDIO_FORMATS = {WAV}
  Audio file formats the module accepts.

This module also uses the auxiliary constants MIN AUDIO LEVEL (−30 dBFS average) and MAX REC LEN (30 minutes), defined in the SRS.

7.3.2 Exported Access Programs

Name	In	Out	Exceptions
transcribe	audioPath: String	transcript: sequence of RawSegment	InvalidAudioError, ModelUnavailableError

The access programs use the following type:

- RawSegment: a tuple (text: String, start: $\mathbb{R}$, end: $\mathbb{R}$, confidence: $\mathbb{R}$, flagged: $\mathbb{B}$, language: String), representing one displayable chunk of transcribed speech.
  - text is the transcribed speech.
  - start and end give the time range of the chunk in the recording.
  - confidence is a score in $[0, 1]$.
  - flagged is true iff the segment needs clinician review.
  - language is the language the model detected in the chunk.
- The transcript is a sequence of RawSegment(s), ordered by start time.

7.4 Semantics

7.4.1 State Variables

None. The module retains no audio or transcript between calls.

7.4.2 Environment Variables

- audioFile: the audio file on the local file system that is being transcribed.
- whisperModel: the locally installed speech-to-text model that converts a chunk of audio into text with a confidence score.

7.4.3 Assumptions

- The model is installed locally and available before transcribe is called.
- audioPath refers to an audio file of one complete assessment session, supplied by the Transcription Service Interface module.
- The model reports a confidence score for each chunk it transcribes.
- The model reports the language it detects for each chunk, one of the supported assessment languages (Mandarin or English).
- The Transcription Service Interface returns the flags, and the Frontend presents flagged segments to the clinician, who reviews them against the video playback (HA-MLTM1).
- This module does not transmit audio or transcripts; transcription runs entirely on the local device, and storage and transmission are handled by the Database and Encryption modules (HA-MLTM2, NFR**??**).
- The audio file is no longer than MAX_REC_LEN.

7.4.4 Access Routine Semantics

transcribe(audioPath):

- transition: None
- output: A sequence of RawSegment(s).
  - Divide the audio at audioPath into consecutive chunks of CHUNK_DURATION.
  - For each chunk in order, obtain its text, confidence, and language from whisperModel and form a RawSegment with the chunk's time range. Set flagged to the value of `shouldFlag` for that segment and chunk.
  - The output are these RawSegment(s) in time order.
- exception:
  - InvalidAudioError if the file does not exist, is unreadable, is empty, or its format is not in SUPPORTED_AUDIO_FORMATS.
  - ModelUnavailableError if whisperModel cannot be loaded or run.

7.4.5 Local Functions

shouldFlag(s: RawSegment, chunkAudio):

- transition: None
- output: true if $s.confidence <$ CONFIDENCE_THRESHOLD, or the average audio level of chunkAudio is below MIN_AUDIO_LEVEL (the model cannot reliably recognize speech at that level, so text may be invented or speech missed).
- exception: None

8 MIS of Transcription Service Interface Module

The Transcription Service Interface module generates a draft transcript for an assessment recording and manages the clinician's review of it: edits, speaker roles, verification and export (FR??, FR??, FR??, NFR??).

8.1 Module

TranscriptionServiceInterface

8.2 Uses

- ML Transcription Model module: transcribe
- Database module: getRecording, saveSegments, getTranscript, updateSegment, setTranscriptStatus, logAccess

This module is used by the API Gateway module, which passes on the clinician's actions after authorizing them, and by the Report Generation module, which reads transcripts.

8.3 Syntax

8.3.1 Exported Constants

- SPEAKER_ROLES = {System, Child, Parent, Experimenter, Multiple Speakers, Others}
  The roles a clinician can assign to a speaker.
- GENERATION_ATTEMPTS = 3
  Number of times transcription is attempted automatically before a failure is reported, so the clinician does not have to restart it manually.
- EXPORT_FORMATS = {CSV}
  File formats in which annotations can be exported.

This module also relies on the auxiliary constants MAX_REC_LEN (30 minutes) and SYNC TOL (500 ms), defined in the SRS.

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
generateTranscript	assessmentID: String, userID: String, portion: Interval or none	-	AssessmentNotFoundError, RecordingUnavailableError, TranscriptionInProgressError, PortionAlreadyTranscribedError, GenerationFailedError
getTranscript	assessmentID: String, userID: String	transcript: Transcript	AssessmentNotFoundError, NoTranscriptError
editSegmentText	assessmentID: String, segmentID: String, newText: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError, SaveFailedError
assignSpeaker	assessmentID: String, segmentID: String, role: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, InvalidRoleError, TranscriptionInProgressError, SaveFailedError
verifySegment	assessmentID: String, segmentID: String, userID: String	-	NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError, SaveFailedError
completeTranscript	assessmentID: String, userID: String	-	NoTranscriptError, TranscriptionInProgressError, SaveFailedError
exportAnnotations	assessmentID: String, userID: String, format: String	file: Bytes	NoTranscriptError, UnsupportedFormatError

The access programs use the following types:

- Interval: a pair of times (from, to) in seconds on the recording's timeline.
- Transcript: a pair (status, segments).
  - status is one of {BEING_GENERATED, DRAFT, VERIFIED}.
  - segments is a sequence of Segment(s), ordered by start time.
- Segment: a tuple (segmentID: String, start: $\mathbb{R}$, end: $\mathbb{R}$, text: String, originalText: String, speaker: String or none, language: String, confidence: $\mathbb{R}$, flagged: $\mathbb{B}$, verified: $\mathbb{B}$).

  - Times are stored to the nearest tenth of a second or better so playback can be synchronized within SYNC_TOL (NFR??).
  - A segment counts as edited when text differs from originalText, so automatic and clinician-modified content can be told apart (NFR??).

The type Recording is defined by the Video Recording module (VideoRecording.Recording) and RawSegment by the ML Transcription Model module.

8.4 Semantics

8.4.1 State Variables

None. All transcript data is held by the Database module, so work saved in one session is available in the next (NFR??).

8.4.2 Environment Variables

None.

8.4.3 Assumptions

- The API Gateway has called the Account System's authorize routine, and userID is the resulting identity of a clinician assigned to the patient of the assessment (NFR??).
- Each assessment has one recording, up to MAX_REC_LEN long, retrievable with Database.getRecording.
- The temporary audio file created by generateTranscript is kept in a protected folder on the server that only this module can read, and is deleted when generation ends.
- Clinicians review flagged segments against the video playback before relying on the transcript (HA-MLTM1).
- Encryption of recordings and transcripts in storage is provided by the Database and Encryption modules, and in transit by the API Gateway (NFR??).

8.4.4 Access Routine Semantics

generateTranscript(assessmentID, userID, portion):

- transition:
  - Database.setTranscriptStatus(assessmentID, BEING_GENERATED).
  - Obtain the recording with Database.getRecording(assessmentID) and extract the audio of the whole recording, or of portion if given, into a temporary audio file (`extractAudio`).

  - Call ML Transcription Model.transcribe on the audio file, retrying up to GENERATION_ATTEMPTS times if the model is unavailable.
  - Convert each returned RawSegment into a Segment: text and originalText are the generated text; confidence, flagged and language are copied; speaker is none; verified is false; and the times are shifted by the start of portion.
  - Database.saveSegments(assessmentID, the segments), then Database.setTranscriptStatus(assessmentID, DRAFT).
  - Database.logAccess(userID, GENERATE, assessmentID).
  - Delete the temporary audio file, whether or not generation succeeded.
  - If generation fails, no segments have been saved, and the status is set back to what it was before the call.
- output: None
- exception: AssessmentNotFoundError if assessmentID does not exist; RecordingUnavailableError if the recording cannot be obtained; TranscriptionInProgressError if the status is already BEING_GENERATED; PortionAlreadyTranscribedError if existing segments overlap a portion (or any segments exist, when portion is none); GenerationFailedError if all attempts fail.

getTranscript(assessmentID, userID):

- transition: Database.logAccess(userID, VIEW, assessmentID).
- output: The transcript returned by Database.getTranscript(assessmentID), including its status (so the caller can tell the clinician that it is still being generated).
- exception: AssessmentNotFoundError if assessmentID does not exist; NoTranscriptError if no transcript has been generated for it.

editSegmentText(assessmentID, segmentID, newText, userID):

- transition: Database.updateSegment(assessmentID, segmentID, text := newText), then Database.logAccess(userID, EDIT, assessmentID). originalText is unchanged.
- output: None
- exception: NoTranscriptError, SegmentNotFoundError, TranscriptionInProgressError if the status is BEING_GENERATED; SaveFailedError if the change could not be saved, in which case nothing is changed and the caller keeps the clinician's input to retry.

assignSpeaker(assessmentID, segmentID, role, userID):

- transition: Database.updateSegment(assessmentID, segmentID, speaker := role), then Database.logAccess(userID, EDIT, assessmentID).

- output: None
- exception: InvalidRoleError if role is not in SPEAKER_ROLES; otherwise as for editSegmentText.

verifySegment(assessmentID, segmentID, userID):

- transition: Database.updateSegment(assessmentID, segmentID, verified := true), then Database.logAccess(userID, EDIT, assessmentID).
- output: None
- exception: Same as editSegmentText.

completeTranscript(assessmentID, userID):

- transition: Database.setTranscriptStatus(assessmentID, VERIFIED), then Database.logAccess(userID, EDIT, assessmentID). Segments that are not verified keep verified false; the caller finds them through getTranscript.
- output: None
- exception: NoTranscriptError, TranscriptionInProgressError, SaveFailedError.

exportAnnotations(assessmentID, userID, format):

- transition: Database.logAccess(userID, EXPORT, assessmentID).
- output: A file in the given format containing, for every segment of the transcript returned by Database.getTranscript(assessmentID), its start and end times, text, speaker, language and verified value (FR??).
- exception: NoTranscriptError if no transcript exists; UnsupportedFormatError if format is not in EXPORT_FORMATS.

8.4.5 Local Functions

extractAudio(recording, portion):

- transition: Writes a temporary audio file.
- output: A file in a format supported by the ML Transcription Model containing the audio of recording (restricted to portion, if given).
- exception: RecordingUnavailableError if the recording has no readable audio.

9 MIS of Video Recording Module

The Video Recording module checks recording quality, records an assessment session through the patient's camera and microphone, and ensures the recording is saved intact and backed up (NFR??, NFR??).

9.1 Module

VideoRecording

9.2 Uses

- Device I/O module: sample, startCapture, stopCapture, saveLocal, clearLocal
- API Gateway module: uploadRecording, requestBackup

This module is used by the Frontend module during an assessment. Storage, encryption and backup of the uploaded recording are handled by the Database module behind the API Gateway.

9.3 Syntax

9.3.1 Exported Constants

- SAMPLE_DURATION = 5 seconds
  Length of the sample taken from the camera and microphone for the quality check.
- MIN_BRIGHTNESS = 0.2
  Minimum average video brightness, on a scale from 0 (black) to 1 (white), for the lighting to be considered adequate.
- MIN_FRAME_RATE = 15 frames per second
  Minimum video frame rate accepted.
- MAX_UPLOAD_ATTEMPTS = 3
  Number of times uploading is attempted automatically before the failure is reported.

This module also uses the auxiliary constants MIN_AUDIO_LEVEL (-30 dBFS average), MIN_VIDEO_RESOLUTION (640×480 pixels) and MAX_REC_LEN (30 minutes), defined in the SRS.

9.3.2 Exported Access Programs

Name	In	Out	Exceptions
init	-	-	-
checkQuality	-	report: QualityReport	PermissionDeniedError, DeviceUnavailableError
startRecording	-	-	PermissionDeniedError, DeviceUnavailableError, RecordingInProgressError
stopRecording	-	-	NotRecordingError, StorageFullError
saveRecording	assessmentID: String, sessionToken: String	-	NoRecordingError, UnauthorizedError, UploadFailedError, VerificationFailedError, BackupFailedError
getStatus	-	status: {IDLE, RECORDING, PENDING_SAVE, SAVED}	-

The access programs use the following types:

- QualityReport: a tuple (passed: $\mathbb{B}$, issues: sequence of String).
  - passed is true iff the audio and video meet the minimum levels above.
  - issues describes each problem found (e.g., audio too quiet, lighting too dark, resolution too low) so the patient can correct their setup.
- Recording: a tuple (file: Bytes, format: String, duration: $\mathbb{R}$ seconds, resolution: $\mathbb{N} \times \mathbb{N}$, recordedAt: Time). This type is passed to the API Gateway and read back by the Transcription Service Interface through the Database.

The type MediaSample is defined by the Device I/O module.

9.4 Semantics

9.4.1 State Variables

- status: {IDLE, RECORDING, PENDING_SAVE, SAVED}
  Where the current recording is in its lifecycle.
- localCopy: Recording or none
  The temporary copy of the current recording kept on the patient's device (through Device I/O) until the system confirms it was saved successfully.

9.4.2 Environment Variables

None. The camera, microphone and local storage are reached only through the Device I/O module.

9.4.3 Assumptions

- init is called before any other access program.
- The caller runs checkQuality, and the patient has corrected any reported issues, before startRecording is called (HA-VR1).
- sessionToken identifies a logged-in user who is allowed to upload a recording for assessmentID; this is checked by the API Gateway.
- The patient's device has enough local storage to hold the full recording.

9.4.4 Access Routine Semantics

init():

- transition: status := IDLE; localCopy := none.
- output: None
- exception: None

checkQuality():

- transition: None
- output: A QualityReport based on Device I/O.sample(SAMPLE_DURATION). passed is true iff the sample's audio level is at least MIN_AUDIO_LEVEL, its brightness is at least MIN_BRIGHTNESS, its frame rate is at least MIN_FRAME_RATE, and its resolution is at least MIN_VIDEO_RESOLUTION. Each failed condition adds an entry to issues. The sample is discarded.
- exception: PermissionDeniedError if access to the camera or microphone is not granted; DeviceUnavailableError if either device is missing or in use elsewhere.

startRecording():

- transition: Device I/O.startCapture(); status := RECORDING. Recording stops automatically when its length reaches MAX_REC_LEN.
- output: None
- exception: PermissionDeniedError, DeviceUnavailableError (from Device I/O); RecordingInProgressError if status = RECORDING.

stopRecording():

- transition: recording := Device I/O.stopCapture(); Device I/O.saveLocal(recording); localCopy := recording; status := PENDING_SAVE.
- output: None
- exception: NotRecordingError if status ≠ RECORDING; StorageFullError if the recording cannot be kept on the device.

saveRecording(assessmentID, sessionToken):

- transition:
  - Call API Gateway.uploadRecording(sessionToken, assessmentID, localCopy), up to MAX_UPLOAD_ATTEMPTS times, to obtain the checksum of the stored recording.
  - Check the result with `isIntact`.
  - If it passes, call API Gateway.requestBackup(sessionToken, assessmentID), then Device I/O.clearLocal(); localCopy := none; status := SAVED.
  - If any step fails, localCopy and status are unchanged, so saveRecording can be called again.
- output: None
- exception: NoRecordingError if status ≠ PENDING_SAVE; UnauthorizedError if the session is not valid for this assessment; UploadFailedError if uploading did not succeed after MAX_UPLOAD_ATTEMPTS attempts; VerificationFailedError if the stored recording does not match localCopy; BackupFailedError if the backup request fails.

getStatus():

- transition: None
- output: status
- exception: None

9.4.5 Local Functions

isIntact(checksum: String, recording: Recording):

- transition: None
- output: true iff checksum equals the checksum computed from recording, and recording can be decoded and played from start to end.
- exception: None

10 Appendix — Generative AI Use Disclosure

This section documents all the uses of generative artificial intelligence used throughout the document.

10.1 AI Tools Used

- Claude Sonnet 5 (Medium)
- ChatGPT-5.6 Luna

10.2 How and Where Used

The following lists the uses of AI applied throughout the entire document:

- Fixing grammar and spelling
- Debugging possible LaTeX errors
- Performing a final check over the document at the end, comparing each section against the original document comments and the rubric to ensure nothing was missed
- Removing template comments

Section 5

- Format the table of existing modules for LaTeX

All Modules

- Brainstorm modules in addition to the already existing ones
- Flesh out ideas for each module

10.3 Verification of AI-Generated Content

To validate the LLM's output, each team member reviewed their assigned section and presented it to the group, summarizing the work they had completed since the last meeting. All members were able to explain their sections clearly. Before any pull request was approved, a team member who had not authored the section reviewed it to confirm that it was cohesive, accurate, and consistent with the rest of the document. Because the document contained no factual claims, the LLM's output did not require any additional fact-checking.

11 Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

7. (After you have implemented another team's module, which means this isn't filled in until after the original deadline). What did you learn by implementing another team's module? Were all the details you needed in the documentation, or did you need to make assumptions, or ask the other team questions? If your team also had another team implement one of your modules, what was this experience like? Are there things in your documentation you could have changed to make the process go more smoothly for when an "outsider" completes some of the implementation?

Your reflection should be about concepts learned in your SE education, such as modular decomposition, cohesion, coupling, information hiding, etc. You should critique the quality and utility of the specification you received.

You should create a PR for your implemented module. A link to the PR should be part of your reflection.

Nour

Question 1

Writing the MIS went smoothly because we already had a clear understanding of what each module is meant to do and how it contributes to the system. Our requirements and hazard analysis gave us a solid starting point, so we could focus on defining each module's interface instead of deciding what the module was for. Knowing each module's intended behaviour made it easier to decide what to expose through its access programs and what to keep hidden.

Question 2

The most difficult part was figuring out how the different modules would interact considering several of them depend on each other. For example, the Transcription Service Interface relies on Video Storage, the ML Transcription Model and the Database. It was hard to decide which module should own each responsibility and where one module's interface should end and the next begin. We resolved this by discussing the data that passes between modules and keeping each interface to the minimum its callers need. This kept coupling low and let us write each module without knowing how the others work internally.

Question 3

The ML Transcription Model decision came from our supervisors. They already use Whisper in their existing workflow, but they access it through an API. However, they required us to implement the system such that the model is run locally. This changes the module's interface and means recordings do not have to leave the device for transcription.

The decomposition of the remaining modules were our own decisions. We split them so that each module has a single responsibility, without breaking the system down so far that the modules became trivial. I worked on the Video Recording and Transcription Service Interface modules. We made each its own module because they handle different concerns. Video Recording deals with capturing and verifying recordings, and the Transcription Service Interface manages generating and reviewing transcripts.

Question 4

None of our other documents really needed to be changed. In fact, we referenced our requirements and hazard analysis while deciding on the different modules of the system, hence they were useful as is. Our supervisors also checked over our requirements and were satisfied with them. That told us we were on the right track, so we went ahead and started designing our system.

Question 5

The biggest limitation is scope. Some features in our requirements document, such as result visualization and patient-facing reports, are listed under likely changes. They are low priority, so we don't expect to build them within this project and have documented them as future work.

The other limitation is the transcription model itself. We use Whisper, and its performance is average, particularly for difficult Mandarin words. We reduced the risk by flagging low-confidence segments and having the clinician review and correct the transcript. It still means the transcripts won't always be accurate on the first pass.

With unlimited time and resources, we would first add an interference detection system using a machine learning model. We would also evaluate how accurate the transcription is on Mandarin-English speech, for example by comparing the generated transcripts with the clinicians' corrected versions.

Question 6

The first alternative design we thought of was combining recording, transcription and storage into one large module. The benefit would be fewer interfaces to manage, and there would be no need to decide how modules communicate. However, the trade-off is that the module would have low cohesion, so a change in one area (for example, transcript editing, which we expect to change) could affect the others. We didn't choose it because separate modules let each one hide its own design decisions and be tested on its own.

The second alternative design was having the transcription module keep the transcript in memory. This would be simpler to write; however, edits could be lost between sessions, and clinicians might not be able to continue their work from another device. We didn't choose it because our transcription module stores nothing itself and saves all transcript data through the Database module, so the saved work is preserved.

Overall, we chose our design because it separates responsibilities cleanly, so each module can be changed and tested on its own. This also allows each module to be small enough to build in the time we have.